

Project 2 Documentation

CS 643

Vivek Shah

December 7th, 2025

Table of Contents

0. Prerequisites & Project Artifacts.....	3
0.1 Required Files.....	3
0.2 Other Requirements.....	3
0.3 Links.....	3
0.4 ChatGPT.....	3
1. Log in to AWS Academy Learner Lab & Open Console.....	4
2. Create & Prepare an S3 Bucket.....	5
3. Create the EMR Cluster (Parallel Training).....	6
4. Run Parallel Model Training on EMR.....	8
4.1 Get the EMR Master Public DNS & SSH Command.....	8
4.2 SSH into EMR Primary.....	8
4.3 Copy the Training Script from S3.....	9
4.4 Run Spark Training with winePrediction_model.py.....	9
4.5 Verify Model Artifacts in S3.....	10
5. Launch a Standalone EC2 Instance for Prediction.....	11
5.1 Create EC2 Instance.....	11
5.2 SSH into the EC2 Instance.....	11
6. Configure the EC2 Instance for Prediction (PySpark).....	12
6.1 Install System Dependencies.....	12
6.2 Install Python Libraries.....	12
6.3 Download Model and Application from S3.....	12
7. Run the Prediction Application on EC2.....	13
7.1 Run with spark-submit.....	13
8. Dockerized Prediction Application.....	14
8.1 Create Required Files & Build Docker Image.....	14
8.2 Test the Docker Container.....	15
8.3 Push Docker Container to Docker Hub.....	15
9. Important Files.....	16
9.1 Dockerfile.....	16
9.2 entrypoint.sh.....	17

0. Prerequisites & Project Artifacts

0.1 Required Files

Please have the following files saved locally in a known location:

- TrainingDataset.csv
- ValidationDataset.csv
- winePrediction_model.py - used for Spark training on EMR cluster
- winePrediction_application.py - used Spark inference on EC2 instance or Docker container

0.2 Other Requirements

- Access to AWS, AWS Academy Learner Lab, or equivalent cloud environment
- AWS Region
- A key pair (.pem file) for SSH access to EMR and EC2

0.3 Links

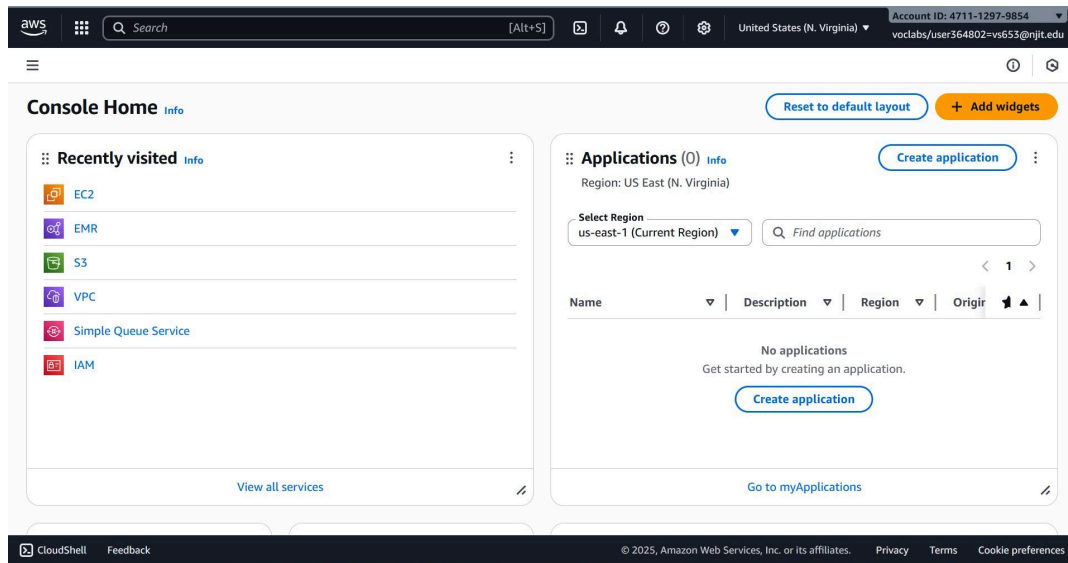
- GitHub: <https://github.com/vs653/cs643-853-pa2-vs653>
- DockerHub: <https://hub.docker.com/repository/docker/vs653njit/wine-predictor/general>

0.4 ChatGPT

ChatGPT was used in this project to break down the project requirements and create a list of objectives to complete the project. This helped me build a timeline and workflow for the project. The code used in this project was written by me, however, in the case of errors, I used ChatGPT to troubleshoot those errors and find a way to remediate it. It was helpful diagnosing errors and providing multiple solutions to try.

1. Log in to AWS Academy Learner Lab & Open Console

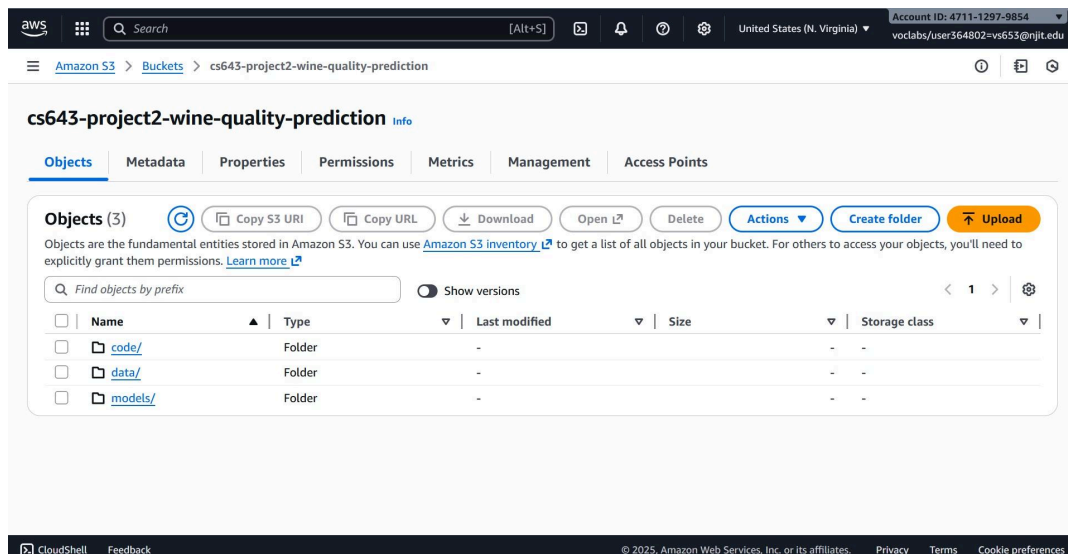
1. Log into AWS Academy Learner Lab or AWS
2. Start Lab session
3. Save SSH key pair (.pem file)
4. Click the button to Open AWS Console
5. Confirm Region, located in the top-right. Ex: N. Virginia (us-east-1)



2. Create & Prepare an S3 Bucket

We are going to store the datasets, scripts, and the model in a S3 bucket so both the EMR cluster and EC2 instances can access them.

1. In the AWS Console, go to Services, then S3
2. Click “Create bucket”
3. Select “General Purpose”
4. Choose a unique Bucket name, ex: “cs643-project2-wine-prediction”
5. Keep the remaining settings to their default setting
6. Click “Create bucket”
7. Open the new bucket and create the following folders:
 - a. data
 - b. code
 - c. models
8. Upload the following files to their specified locations:
 - a. TrainingDataset.csv - data
 - b. ValidationDataset.csv - data
 - c. winePrediction_model.py - code
 - d. winePrediction_application.py - code



3. Create the EMR Cluster (Parallel Training)

We will use an EMR cluster with multiple CORE nodes so Spark can train the model in parallel.

1. In AWS Console go to Services, then EMR
2. Click “Create cluster”

Here are the key settings and the value to set them each to:

- Name and applications:
 - Cluster Name, ex: wine-prediction-training-cluster
 - Release: emr-7.12.0
 - Application Bundle: Spark Interactive
 - Make sure Spark is included in the list of applications
 - Operating System Options: Amazon Linux Release
 - Cluster Configuration:
 - Uniform Instance Groups
 - Primary - EC2 Instance Type: m5.xlarge
 - Core - EC2 Instance Type: m5.xlarge
 - Remove All Task Instance Groups
 - Provisioning Configuration: Core - 4
 - Networking
 - Leave as defaults, except EMR managed security group
 - Click on the security group for the EMR managed security group
 - Add 3 Inbound Rules - SSH, HTTP, HTTPS
 - Update “Source” to “My IP” for these 3 new rules
 - Security Configuration and EC2 Key Pair
 - Amazon EC2 key pair for SSH to the cluster: vockey (or name of .pem file downloaded)
 - Identity and Access Management (IAM) roles
 - Choose an existing instance profile
 - Service role: EMR_DefaultRole
 - Choose an existing instance profile
 - Instance profile: EMR_EC2_DefaultRole
3. Scroll to the bottom and click “Create cluster”
 4. Wait for the cluster status to go from: “STARTING” to “BOOTSTRAPPING” to “WAITING”

aws

Search

[Alt+S]

United States (N. Virginia)

Account ID: 4711-1297-9854
voclabs/user364802-vs653@njit.edu

Amazon EMR

EMR on EC2: Clusters

CS643_Project2_Cluster

Core 4 instances

Instance groups (2) Info

With the instance groups configuration, each node type consists of the same instance type and the same purchasing option for instances: On-Demand or Spot.

Find instances by status Find resources by ID or type; or search for text within loaded results

Type and name	ID	Status	Instances	Purchasing option and p...	EBS size (GiB)
Primary m5.xlarge 4 vCore, 16 GiB memory, EBS only stor...	ig-35HF786E7GUWI	Bootstrapping	0 (1 requested)	On-Demand	-
Core m5.xlarge 4 vCore, 16 GiB memory, EBS only stor...	ig-082816034K5FHG4...	Bootstrapping	-	On-Demand (-)	64 (2 volumes)
m5.xlarge 4 vCore, 16 GiB memory, EBS only stor...	ig-L0O0N71Q5SMA	Provisioning	0 (4 requested)	On-Demand	-
m5.xlarge 4 vCore, 16 GiB memory, EBS only stor...	ci-05673981TAFN1R1...	Bootstrapping	-	On-Demand (-)	64 (2 volumes)
m5.xlarge 4 vCore, 16 GiB memory, EBS only stor...	ci-05053632249JTOK...	Bootstrapping	-	On-Demand (-)	64 (2 volumes)
m5.xlarge 4 vCore, 16 GiB memory, EBS only stor...	ci-093939522RNX3O...	Bootstrapping	-	On-Demand (-)	64 (2 volumes)
m5.xlarge 4 vCore, 16 GiB memory, EBS only stor...	ci-06721213OM2SP7...	Bootstrapping	-	On-Demand (-)	64 (2 volumes)

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws

Search

[Alt+S]

United States (N. Virginia)

Account ID: 4711-1297-9854
voclabs/user364802-vs653@njit.edu

Amazon EMR

EMR on EC2: Clusters

Amazon EMR

EMR Serverless

EMR on EC2 Clusters

Notebooks and Git repos

Events

Block public access

Security configurations

EMR on EKS Virtual clusters

EMR Studio Getting Started

Studios

Workspaces (Notebooks)

What's New

Video tour

Clusters (2) Info

Filter clusters by status Find clusters

Filter clusters by creation date-time

Cluster ID	Cluster name	Status	Creation time (UTC-05:00)
i-2V8RHGGL3SHC1	CS643_Project2_Cluster	Starting Preparing cluster	November 30, 2025, 22:24
i-32LJPF9YBUIU7	CS643_Project2_Cluster	Terminated User request	November 23, 2025, 18:06

Steps (0)

Failed 0

Running 0

Completed 0

Cancelled 0

Pending 0

Cancel pending 0

Interrupted 0

Add step

Instances (5)

Primary: (bootstrapping) 1 m5.xlarge

Core: (provisioning) 4 m5.xlarge

Applications (7)

Hadoop 3.4.1

Hive 3.1.3

JupyterEnterpriseGateway 2.6.0 +4

Quick links

Bootstrap actions

Configurations

Monitoring

Events

Primary node public DNS

ec2-3-235-163-104.compute-1.amazonaws.com

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

4. Run Parallel Model Training on EMR

Once created, we will SSH into the primary node of the EMR cluster, copy the training script, and run Spark.

4.1 Get the EMR Master Public DNS & SSH Command

1. In the EMR console, click on the cluster you created.
2. On the Cluster details page, find:
 - a. Primary Node - Public DNS
3. Copy this DNS for later use

4.2 SSH into EMR Primary

4. On your local machine, SSH into the EMR primary using the following command:
 - a. `ssh -i vockey.pem hadoop@<EMR_Public_DNS>`
5. Type “yes” to accept the host key
6. Create a directory for storing AWS credentials
7. Create credentials file - paste AWS credentials (copied from AWS Learner Lab console)

[illegible]

4.3 Copy the Training Script from S3

8. Once logged into the EMR primary node, make the directory for your project
9. Copy the training script from the S3 bucket

[illegible]

4.4 Run Spark Training with winePrediction_model.py

We will read the data from S3 and write the ensemble model back to S3

10. Execute the highlighted command

```
[hadoop@ip-172-31-3-54: ~]$ cd /s3/wine_project
[hadoop@ip-172-31-3-54: ~]$ cd /s3/wine_project
[hadoop@ip-172-31-3-54: /s3/wine_project]$ spark-submit minePredictionModel.py \
s3://cs643-projects2-wine-quality-prediction/data/TrainingDataset.csv \
s3://cs643-projects2-wine-quality-prediction/data/ValidationDataset.csv \
s3://cs643-projects2-wine-quality-prediction/models/winePredictionModel_model_ensemble
25/12/01 04:34:39 INFO FileTransferChannel: Setting File mode to raise
25/12/01 04:34:38 INFO SparkContext: Running Spark version 3.5.6-amzn-1
25/12/01 04:34:38 INFO SparkContext: OS info Linux, 6.1.156-1728.anz2023.x86_64, amd64
25/12/01 04:34:38 INFO SparkContext: Java version 17.0.17
25/12/01 04:34:38 INFO ResourceUtils: =====
25/12/01 04:34:38 INFO ResourceUtils: No custom resources configured for spark.driver.
25/12/01 04:34:38 INFO ResourceUtils: =====
25/12/01 04:34:38 INFO SparkContext: Default application WinQuality_P2_Ensemble_Testing_v3
25/12/01 04:34:38 INFO ResourceProfile: Default ResourceProfile created, executor resources: Map(executorType -> name: executorType,
amount: 1, script: , vendor: , cores -> name: cores, amount: 4, script: , vendor: , memory -> name: memory, amount: 9486, script: , v
endor: , offheap -> name: offheap, amount: 0, script: , vendor: ), task resources: Map(cpu -> name: cpu, amount: 1.0)
25/12/01 04:34:38 INFO ResourceProfileManager: Added ResourceProfile id: 0
25/12/01 04:34:38 INFO ResourceProfile: Limiting resource is cpu at 4 tasks per executor
25/12/01 04:34:38 INFO ResourceProfileManager: Added ResourceProfile id: 0
25/12/01 04:34:38 INFO ResourceProfile: User executor ResourceProfile created, executor resources: Map(executorType -> name: executor
Type, amount: 1, script: , vendor: , cores -> name: cores, amount: 4, script: , vendor: , memory -> name: memory, amount: 9486, scrip
t: , vendor: , offheap -> name: offheap, amount: 0, script: , vendor: ), task resources: Map(cpu -> name: cpu, amount: 1.0)
25/12/01 04:34:38 INFO ResourceProfileManager: Added ResourceProfile id: 1
25/12/01 04:34:38 INFO ResourceProfile: Limiting resource is cpu at 4 tasks per executor
25/12/01 04:34:38 INFO ResourceProfileManager: Added ResourceProfile id: 1
25/12/01 04:34:38 INFO SecurityManager: Changing view acls to: hadoop
25/12/01 04:34:38 INFO SecurityManager: Changing modify acls to: hadoop
25/12/01 04:34:38 INFO SecurityManager: Changing view acls groups to:
25/12/01 04:34:38 INFO SecurityManager: Changing modify acls groups to:
25/12/01 04:34:38 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: hadoo
p groups with view permissions: EMPTV; users with edit permissions: hadoop; groups with modify permissions: EMPTV
25/12/01 04:34:39 INFO Utils: Successfully started service 'sparkDriver' on port 404663.
25/12/01 04:34:39 INFO SparkEnv: Registering MapOutputTracker
25/12/01 04:34:39 INFO SparkEnv: Registering BlockManagerMaster
25/12/01 04:34:39 INFO BlockManagerMasterEndpoint: Using org.apache.spark.storage.DefaultTopologyMapper for getting topology informat
ion
25/12/01 04:34:39 INFO BlockManagerMasterEndpoint: BlockManagerMasterEndpoint up
```

This script will:

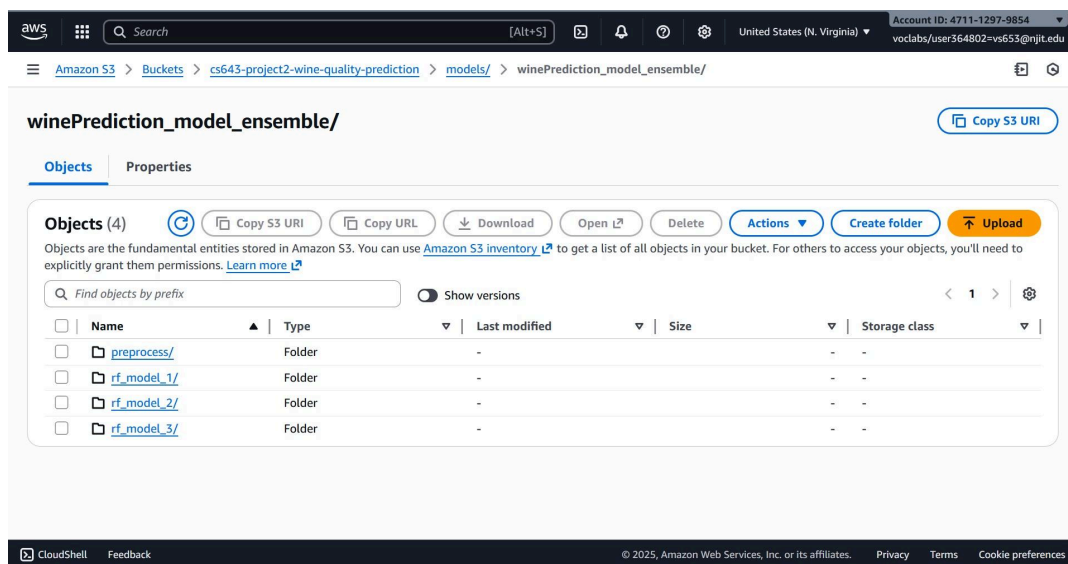
- Load and preprocess the training and validation data
- Fit a preprocessing pipeline (VectorAssembler + Standard Scaler) on train
- Add class weights based on training distribution
- Train 3 different RandomForest classifiers in parallel
- Combine them via majority vote or rounded-average tie-breaker

- Report individual RF F1 scores and ensemble F1
- Save created models: preprocess, RandomForest 1-3

4.5 Verify Model Artifacts in S3

Navigate Back to S3 bucket, under the “models” folder you should see:

- preprocess/
- rf_model_1/
- rf_model_2/
- rf_model_3/



5. Launch a Standalone EC2 Instance for Prediction

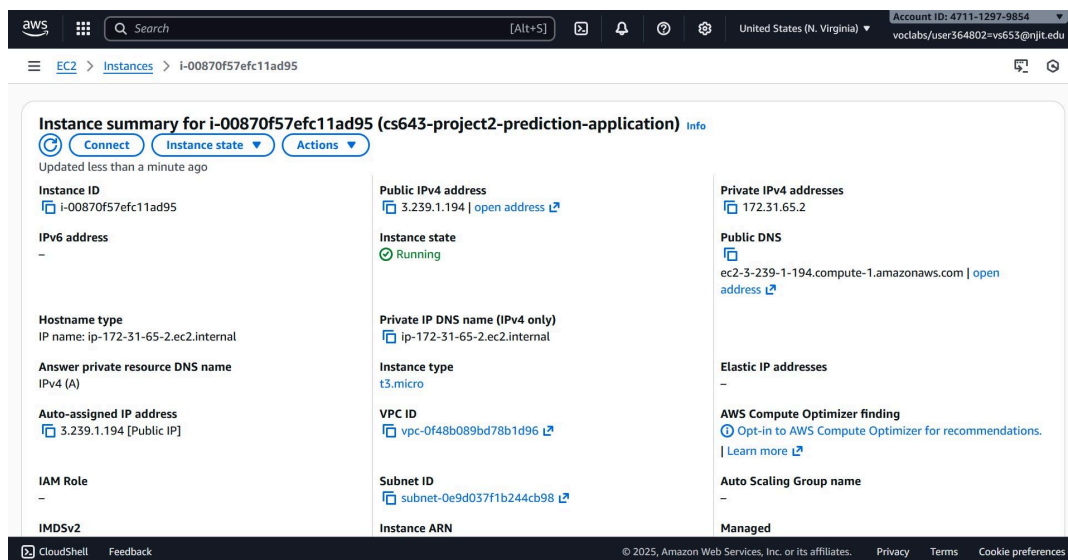
After creating and training the prediction model, we will set up a separate EC2 instance to run the prediction application.

5.1 Create EC2 Instance

1. In AWS console, go to Services, then to EC2, then Instances
2. Click “Launch Instance”

Set Key Settings:

- Name: wine-prediction-ec2
- AMI: Amazon Linux 2023 kernel-6.1 AMI
- Instance Type: t3.medium
- Key Pair: use same key pair as EMR - vockey.pem
- Network:
 - Leave VPC as default
 - Create Security Group - Allow SSH, HTTPS, HTTP traffic from “My IP”



5.2 SSH into the EC2 Instance

3. From your local machine, SSH into the EC2 instance:
 - a. `ssh -i vockey.pem ec2-user@<EC2_Public_IP>`

6. Configure the EC2 Instance for Prediction (PySpark)

After creating the EC2 instance, configure the EC2 instance for the prediction application. We will set up Java, Python, and PySpark so winePrediction_application.py can run with spark-submit in local mode.

6.1 Install System Dependencies

1. On the EC2 Instance, update packages, install Java, Python 3, and pip

Commands:

```
sudo yum update -y
sudo yum install -y java-17-amazon-corretto-headless
sudo yum install -y python3 python3-pip
```

6.2 Install Python Libraries

1. On the EC2 Instance, install PySpark libraries

Commands:

```
pip3 install --user pyspark==3.5.0 numpy pandas
```

6.3 Download Model and Application from S3

Create a project folder and sync the model and validation data.

1. Executed the highlighted commands:

```
ec2-user@ip-172-31-65-2 ~/$ mkdir -p ~/wine_app
ec2-user@ip-172-31-65-2 ~/$ cd ~/wine_app
ec2-user@ip-172-31-65-2 wine_app$ aws s3 cp s3://cs643-project2-wine-quality-prediction/code/winePrediction_application.py .
download: s3://cs643-project2-wine-quality-prediction/code/winePrediction_application.py to ./winePrediction_application.py
ec2-user@ip-172-31-65-2 wine_app$ aws s3 cp s3://cs643-project2-wine-quality-prediction/data/ValidationDataset.csv .
download: s3://cs643-project2-wine-quality-prediction/data/ValidationDataset.csv to ./ValidationDataset.csv
ec2-user@ip-172-31-65-2 wine_app$ aws s3 cp s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble ./winePrediction_model_ensemble --recursive
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/preprocess/metadata/_SUCCESS to winePrediction_model_ensemble/preprocess/metadata/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/preprocess/stages/0_VectorAssembler_45fed237e5ef/metadata/_SUCCESS to winePrediction_model_ensemble/preprocess/stages/0_VectorAssembler_45fed237e5ef/metadata/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/preprocess/metadata/part-000000 to winePrediction_model_ensemble/preprocess/metadata/part-000000
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/preprocess/stages/1_StandardScaler_49cd13885e85/metadata/part-000000 to winePrediction_model_ensemble/preprocess/stages/1_StandardScaler_49cd13885e85/metadata/part-000000
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/rf_model_1/metadata/_SUCCESS to winePrediction_model_ensemble/rf_model_1/metadata/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/rf_model_2/metadata/_SUCCESS to winePrediction_model_ensemble/rf_model_2/metadata/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/rf_model_2/treesMetadata/_SUCCESS to winePrediction_model_ensemble/rf_model_2/treesMetadata/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/rf_model_1/treesMetadata/_SUCCESS to winePrediction_model_ensemble/rf_model_1/treesMetadata/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/preprocess/stages/0_VectorAssembler_45fed237e5ef/metadata/part-000000 to winePrediction_model_ensemble/preprocess/stages/0_VectorAssembler_45fed237e5ef/metadata/part-000000
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/preprocess/stages/1_StandardScaler_49cd13885e85/data/_SUCCESS to winePrediction_model_ensemble/preprocess/stages/1_StandardScaler_49cd13885e85/data/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/preprocess/stages/1_StandardScaler_49cd13885e85/metadata/_SUCCESS to winePrediction_model_ensemble/preprocess/stages/1_StandardScaler_49cd13885e85/metadata/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/rf_model_3/data/_SUCCESS to winePrediction_model_ensemble/rf_model_3/data/_SUCCESS
download: s3://cs643-project2-wine-quality-prediction/models/winePrediction_model_ensemble/rf_model_1/treesMetadata/part-00000-d9864331-f862-43e0-ac8c-d367f19ab904-c000.snappy.parquet to winePrediction_model_ensemble/rf_model_1/treesMetadata/part-00000-d9864331-f862-43e0-ac8c-d367f19ab904-c000.snappy.parquet
```

7. Run the Prediction Application on EC2

After downloading necessary files and setting up the EC2 instance, run the ensemble application on the validation dataset.

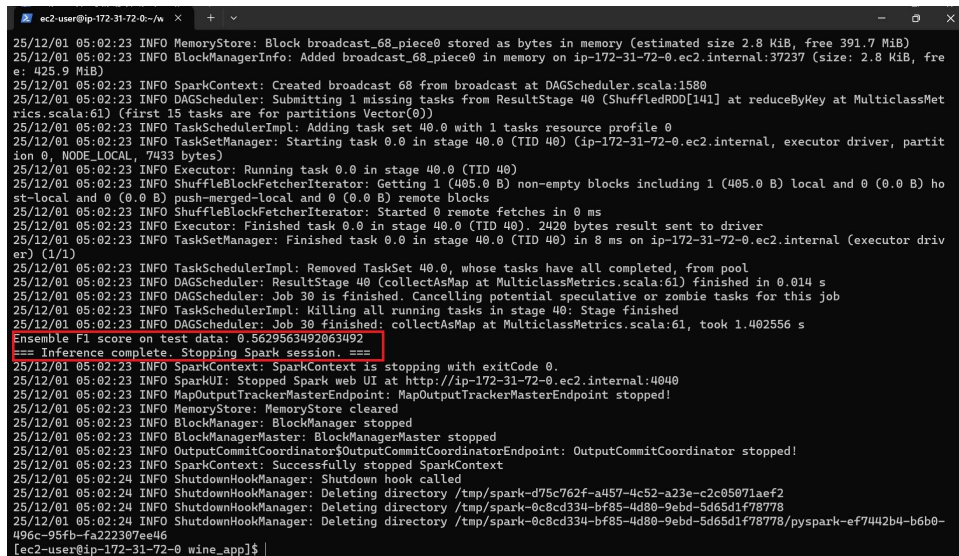
7.1 Run with spark-submit

1. From the wine_app directory execute the following command:

```
spark-submit winePrediction_application.py \  
./ValidationDataset.csv \  
./winePrediction_model_ensemble
```

This command will:

- Load and preprocess the validation data
- Load the preprocessing model and all 3 RandomForest classifier
- Generate predictions from each RandomForest classifier
- Combine them via majority vote or rounded-average tie breaker
- Print Sample of Values: label, prediction from all 3 models, and the ensemble prediction
- Compute and Print Ensemble F1 Score



```
ec2-user@ip-172-31-72-0: ~/wine_app$ spark-submit winePrediction_application.py \
./ValidationDataset.csv \
./winePrediction_model_ensemble

25/12/01 05:02:23 INFO MemoryStore: Block broadcast_68_piece0 stored as bytes in memory (estimated size 2.8 KiB, free 391.7 MiB)
25/12/01 05:02:23 INFO BlockManagerInfo: Added broadcast_68_piece0 in memory on ip-172-31-72-0.ec2.internal:37237 (size: 2.8 KiB, free: 425.9 MiB)
25/12/01 05:02:23 INFO SparkContext: Created broadcast 68 from broadcast at DAGScheduler.scala:1580
25/12/01 05:02:23 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 40 (ShuffledRDD[141] at reduceByKey at MulticlassMetrics.scala:61) (first 15 tasks are for partitions Vector(0))
25/12/01 05:02:23 INFO TaskSchedulerImpl: Adding task set 40.0 with 1 tasks resource profile 0
25/12/01 05:02:23 INFO TaskSetManager: Starting task 0.0 in stage 40.0 (TID 40) (ip-172-31-72-0.ec2.internal, executor driver, partition 0, NODE_LOCAL, 7433 bytes)
25/12/01 05:02:23 INFO Executor: Running task 0.0 in stage 40.0 (TID 40)
25/12/01 05:02:23 INFO ShuffleBlockFetcherIterator: Getting 1 (405.0 B) non-empty blocks including 1 (405.0 B) local and 0 (0.0 B) host-local and 0 (0.0 B) push-merged-local and 0 (0.0 B) remote blocks
25/12/01 05:02:23 INFO ShuffleBlockFetcherIterator: Started 0 remote fetches in 0 ms
25/12/01 05:02:23 INFO Executor: Finished task 0.0 in stage 40.0 (TID 40). 2420 bytes result sent to driver
25/12/01 05:02:23 INFO TaskSetManager: Finished task 0.0 in stage 40.0 (TID 40) in 8 ms on ip-172-31-72-0.ec2.internal (executor driver) (1/1)
25/12/01 05:02:23 INFO TaskSchedulerImpl: Removed TaskSet 40.0, whose tasks have all completed, from pool
25/12/01 05:02:23 INFO DAGScheduler: ResultStage 40 (collectAsMap at MulticlassMetrics.scala:61) finished in 0.014 s
25/12/01 05:02:23 INFO DAGScheduler: Job 30 is finished. Cancelling potential speculative or zombie tasks for this job
25/12/01 05:02:23 INFO TaskSchedulerImpl: Killing all running tasks in stage 40: Stage finished
25/12/01 05:02:23 INFO DAGScheduler: Job 30 finished: collectAsMap at MulticlassMetrics.scala:61, took 1.402556 s
Ensemble F1 score on test data: 0.5629563492863492
=== Inference complete. Stopping Spark session. ===
25/12/01 05:02:23 INFO SparkContext: SparkContext is stopping with exitCode 0.
25/12/01 05:02:23 INFO SparkUI: Stopped Spark web UI at http://ip-172-31-72-0.ec2.internal:4040
25/12/01 05:02:23 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
25/12/01 05:02:23 INFO MemoryStore: MemoryStore cleared
25/12/01 05:02:23 INFO BlockManager: BlockManager stopped
25/12/01 05:02:23 INFO BlockManagerMaster: BlockManagerMaster stopped
25/12/01 05:02:23 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
25/12/01 05:02:23 INFO SparkContext: Successfully stopped SparkContext
25/12/01 05:02:24 INFO ShutdownHookManager: Shutdown hook called
25/12/01 05:02:24 INFO ShutdownHookManager: Deleting directory /tmp/spark-d75c762f-a457-4c52-a23e-c2c05071aef2
25/12/01 05:02:24 INFO ShutdownHookManager: Deleting directory /tmp/spark-0c8cd334-bf85-4d80-9ebd-5d65d1f78778
25/12/01 05:02:24 INFO ShutdownHookManager: Deleting directory /tmp/spark-0c8cd334-bf85-4d80-9ebd-5d65d1f78778/pyspark-ef7442b4-b6b0-496c-95fb-fa222307ee46
[ec2-user@ip-172-31-72-0 wine_app]$
```

8. Dockerized Prediction Application

We will then create a docker container with the necessary components to run the prediction application.

8.1 Create Required Files & Build Docker Image

Navigate Back to S3 bucket, under the “models” folder you should see:

1. Within wine_app directory, create the Dockerfile:

nano Dockerfile

This file will contain the commands to copy over files and models, install Java, PySpark and other required software, and execute a bash script to run the python application.

2. Then create the entrypoint.sh file:

nano entrypoint.sh

This script will take a test dataset file path as an argument and execute the prediction application with the supplied models

3. Set execute permissions on the entrypoint.sh file

chmod +x entrypoint.sh

4. Build the Docker Image:

docker build -t <dockerHub_userName>/wine-predictor:v1 .

```
[ec2-user@ip-172-31-72-0 wine_app]$ nano Dockerfile
[ec2-user@ip-172-31-72-0 wine_app]$ nano entrypoint.sh
[ec2-user@ip-172-31-72-0 wine_app]$ chmod +x entrypoint.sh
[ec2-user@ip-172-31-72-0 wine_app]$ docker build -t vs653njit/wine-predictor:v1 .
[+] Building 57.3s (13/13) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 771B                                0.0s
=> [internal] load metadata for docker.io/library/python:3.10-slim 0.4s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/8] FROM docker.io/library/python:3.10-slim@sha256:c299e10e007017113f9a1f109dd85e7e634fa94589b856e0e87bb22b2b382a2 3.2s
=> => resolve docker.io/library/python:3.10-slim@sha256:c299e10e007017113f9a1f109dd85e7e634fa94589b856e0e87bb22b2b382a2 0.1s
=> => sha256:c299e10e007017113f9a1f109dd85e7e634fa94589b856e0e87bb22b2b382a2 10.37kB / 10.37kB 0.0s
=> => sha256:7c728f59e9f6f9dabfe35efcfe0da7652ebc2deeb6dc65b0d3ac409b5485af6 1.75kB / 1.75kB 0.0s
=> => sha256:6f924957e3d2926cc6706eea8580f83f1ca6d1eed52365c36ceb9734b643e0bd 5.48kB / 5.48kB 0.0s
=> => sha256:0e4bc2bd6656e6e004e3c749af70e5650bac2258243eb0949dea51cb8b7863db 29.78MB / 29.78MB 0.5s
=> => sha256:9793cbb1e51ababd0971146499e99c711ab58bdf823b0226bd9b97b5c9317b92 1.29MB / 1.29MB 0.2s
=> => sha256:d83c3659b1e9bb36fffd122a6efdd5891f83b363fca9054603b1de7962d15931e 13.82MB / 13.82MB 0.5s
=> => sha256:f86ba98c4d8f9c7caf377d5426b6c24a4c45058d795fd1a06c1a7c54b626c084 250B / 250B 0.4s
=> => extracting sha256:0e4bc2bd6656e6e004e3c749af70e5650bac2258243eb0949dea51cb8b7863db 1.4s
=> => extracting sha256:9793cbb1e51ababd0971146499e99c711ab58bdf823b0226bd9b97b5c9317b92 0.1s
=> => extracting sha256:683c3659b1e9bb36fffd122a6efdd5891f83b363fca9054603b1de7962d15931e 0.9s
=> => extracting sha256:f86ba98c4d8f9c7caf377d5426b6c24a4c45058d795fd1a06c1a7c54b626c084 0.0s
=> [internal] load build context                                    0.4s
=> => transferring context: 4.89MB                                    0.3s
=> [2/8] RUN apt-get update && apt-get install -y --no-install-recommends default-jdk-headless && rm -rf /var/lib/apt/l 11.5s
=> [3/8] WORKDIR /app                                              0.0s
=> [4/8] COPY winePrediction_application.py /app/                  0.0s
=> [5/8] COPY winePrediction_model_ensemble /app/winePrediction_model_ensemble 0.1s
=> [6/8] RUN pip install --no-cache-dir pyspark==3.5.0 numpy pandas 38.0s
=> [7/8] COPY entrypoint.sh /app/entrypoint.sh                    0.0s
=> [8/8] RUN chmod +x /app/entrypoint.sh                           0.3s
=> exporting to image                                              3.5s
=> => writing image sha256:186c112d5adlabce574038465013ccc4a0eb744364ef8872741e75f2bdbc120 3.5s
=> => naming to docker.io/vs653njit/wine-predictor:v1              0.0s
[ec2-user@ip-172-31-72-0 wine_app]$
```

8.2 Test the Docker Container

Execute the following command to run the docker container to verify:

```
docker run --rm -v "$(pwd):/data" <dockerHub_userName>/wine-predictor:v1 /data/ValidationDataset.csv
```

This should start the container and run the entrypoint script using the ValidationDataset.csv as the test dataset. It outputs a sample of the labels and predictions as well as an ensemble F1 score.

```
ec2-user@ip-172-31-72-0:~$ ./w
=== Loading test data from: /data/ValidationDataset.csv ===
=== Loading preprocessing model from: /app/winePrediction_model_ensemble/preprocess ===
=== Transforming test data with preprocessing model ===
=== Loading RF ensemble models from: /app/winePrediction_model_ensemble ===
=== Generating predictions from each RF model ===
=== Computing ensemble prediction via majority vote / rounded average ===
=== Sample predictions from ensemble ===
+-----+-----+-----+-----+-----+
|label|pred_rf1|pred_rf2|pred_rf3|ensemble_prediction|
+-----+-----+-----+-----+-----+
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|6.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|7.0|5.0|5.0|5.0|5.0|
|7.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|5.0|5.0|5.0|5.0|5.0|
|7.0|5.0|6.0|6.0|6.0|
|5.0|5.0|5.0|5.0|5.0|
|4.0|5.0|5.0|5.0|5.0|
|6.0|5.0|5.0|5.0|5.0|
+-----+-----+-----+-----+-----+
only showing top 20 rows

=== Evaluating ensemble F1 on test data ===
Ensemble F1 score on test data: 0.5629563492063492
=== Inference complete. Stopping Spark session. ===
[ec2-user@ip-172-31-72-0 wine_app]$
```

8.3 Push Docker Container to Docker Hub

1. Login to your Docker Hub account via command line:

```
docker login
```

You will enter your username and password.

2. Then push your container to Docker Hub

```
docker push <dockerHub_userName>/wine-predictor:v1
```

3. You will find your container at https://hub.docker.com/r/<dockerHub_userName>/wine-predictor

```
[ec2-user@ip-172-31-72-0 ~]$ docker login
Authenticating with existing credentials...
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
[ec2-user@ip-172-31-72-0 ~]$ docker push vs653njit/wine-predictor:v1
The push refers to repository [docker.io/vs653njit/wine-predictor]
fbc55183b858: Pushed
bfc0e8a855964: Pushed
e1157f24d468: Pushed
3369417a681d: Pushed
a20e394c056f: Pushed
bba64e57d392: Pushed
a31ea230a7c1: Pushed
548ba1e7e829: Layer already exists
a8046a14b5f4: Layer already exists
cb4ecf39d967: Layer already exists
78a290c5e58b: Layer already exists
v1: digest: sha256:da9f5263792a10c8ae209c27f709ea286e96599aadaab554c0c73762ed1409b9 size: 2624
```

9. Important Files

9.1 Dockerfile

FROM python:3.10-slim

Install Java (required by PySpark/Spark)

RUN apt-get update && apt-get install -y --no-install-recommends \
 default-jdk-headless \
 && rm -rf /var/lib/apt/lists/*

WORKDIR /app

Copy prediction script and trained model into the image

COPY winePrediction_application.py /app/

COPY winePrediction_model_ensemble /app/winePrediction_model_ensemble

Install Python dependencies: PySpark, NumPy, Pandas

RUN pip install --no-cache-dir \
 pyspark==3.5.0 \
 numpy \
 pandas

Entrypoint script to accept testFilePath and call Python

COPY entrypoint.sh /app/entrypoint.sh

RUN chmod +x /app/entrypoint.sh

ENTRYPOINT ["/app/entrypoint.sh"]

9.2 entrypoint.sh

FROM python:3.10-slim

Install Java (required by PySpark/Spark)

RUN apt-get update && apt-get install -y --no-install-recommends \
 default-jdk-headless \
&& rm -rf /var/lib/apt/lists/*

WORKDIR /app

Copy prediction script and trained model into the image

COPY winePrediction_application.py /app/

COPY winePrediction_model_ensemble /app/winePrediction_model_ensemble

Install Python dependencies: PySpark, NumPy, Pandas

RUN pip install --no-cache-dir \
 pyspark==3.5.0 \
 numpy \
 pandas

Entrypoint script to accept testFilePath and call Python

COPY entrypoint.sh /app/entrypoint.sh

RUN chmod +x /app/entrypoint.sh

ENTRYPOINT ["/app/entrypoint.sh"]